Bre-B: detección de fraude transaccional en tiempo real

Qué construimos, por qué así y qué aprendimos

Samuel Arango · ingesta y streaming
Mateo Sanz Medina · almacenamiento
Nathalia Valentina Cardoza Azuaje · DataOps
Sebastian Andres Medina Cabezas · consumo y gobernanza

Pregunta central del proyecto

Dado el fraude en pagos inmediatos, ¿qué pipeline de datos construiríamos y por qué exactamente así?

Profesor Andrés Sacre Alzate · 2026-2 · Medellín

El problema: en Bre-B el fraude se consuma en minutos

Bre-B en cifras

- Acredita cada transferencia en unos **20 segundos**, 24/7.
- Primer año: **1.700 millones** de transacciones y 36 millones de usuarios con llave.
- Día récord: **10,4 millones** de operaciones, unas **120 por segundo** en promedio.

El dolor

- Las redes de mulas reparten el dinero entre entidades **en minutos**, y el pago inmediato no se reversa.
- La banca analiza por lotes, con cortes de **12 a 24 horas**: la alerta llega cuando el dinero ya salió.

Lo que se necesita

- Decidir **en segundos** por transacción y avisar a la entidad para que retenga los fondos.
- Guardar todo con su linaje para la auditoría (SFC, UIAF).

Usuario	Qué recibe	Qué decide
Entidad financiera	La alerta en su cola SQS	Retener los fondos en la cuenta destino o bloquear la ordenante
Analistas de fraude	Tablero de Metabase y bandeja de alertas	Confirmar redes, reportar a la UIAF y calibrar el umbral
Cumplimiento	SQL sobre Bronze, Silver y Gold	Reconstruir el linaje de una alerta para la SFC

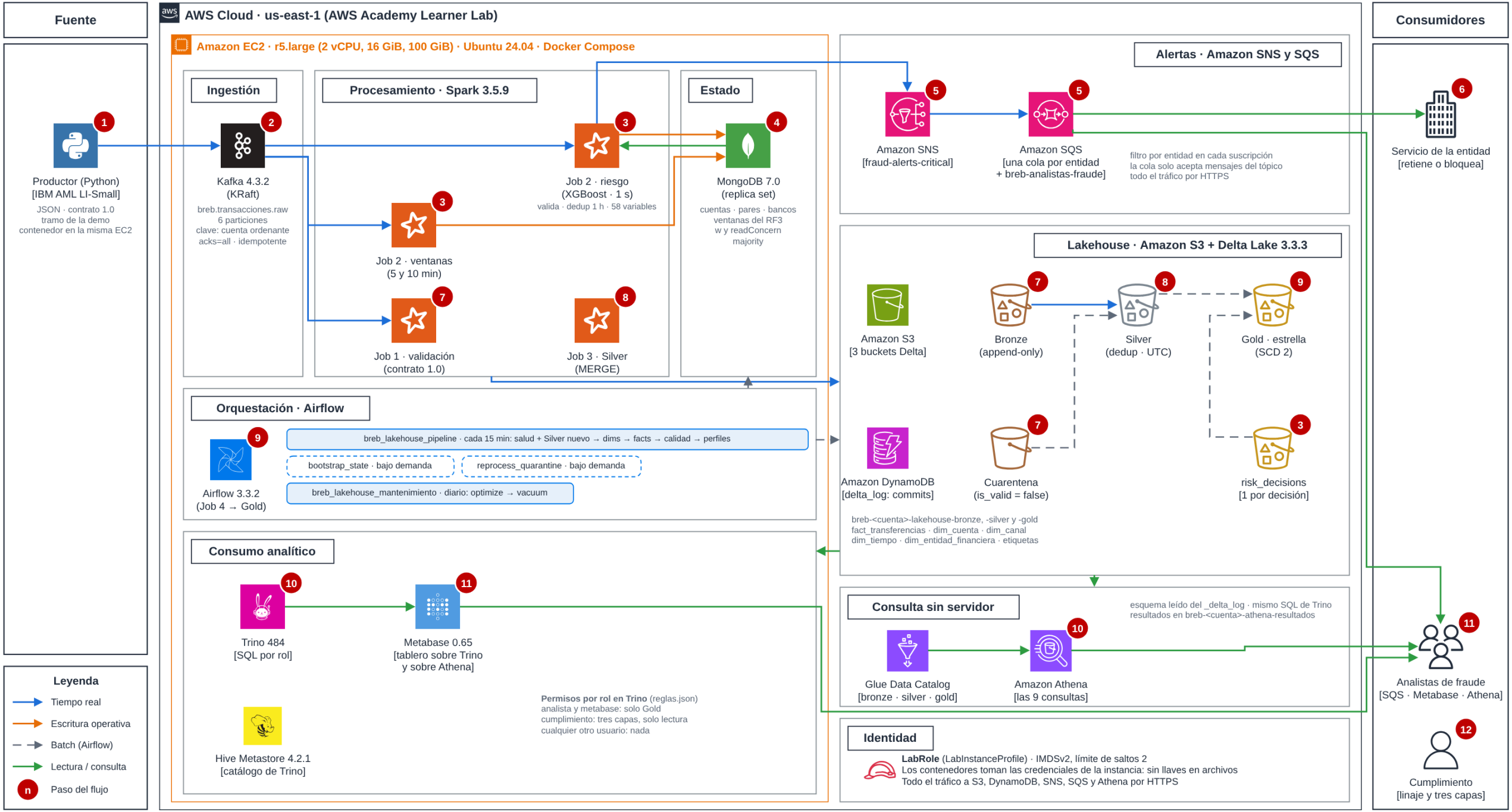
Detección asíncrona con notificación: el sistema no participa en la autorización del pago; decide y avisa en paralelo.

Las cuatro preguntas que responde el sistema

#	Pregunta (Entregable 1)	Cómo se calcula	Dónde se ve
1	¿El ordenante envía a muchos destinatarios nuevos en poco tiempo?	Ventana deslizante de 5 min por ordenante (job2-ventanas) y la variable is_first_sender_pair	cuentas.ventanas en MongoDB · tablero, pregunta 1
2	¿Hay concentración hacia cuentas mula?	Ordenantes distintos y monto recibido por receptora en ventanas de 10 min	cuentas.ventanas · tablero, pregunta 2
3	¿La cuenta reenvía rápido lo que recibe (pass-through)?	Segundos desde la última entrada y fracción reenviada: variables del modelo calculadas con el estado	risk_decisions.variables · tablero, pregunta 3
4	¿Qué tasa de transacciones marca el modelo, por entidad y canal?	Porcentaje de transacciones con score ≥ umbral	Gold: fact_transferencias con dim_canal y dim_entidad_financiera

La decisión la toma el modelo, no las preguntas. XGBoost preentrenado de Kaggle con IBM AML (corrección 2 del profesor): PR-AUC de **0,267** en el test fuera de tiempo, unas 450 veces el de un modelo al azar; umbral 0,172. Las preguntas dan contexto a cada alerta y alimentan los tableros.

Bre-B · Detección de fraude en tiempo real · Arquitectura desplegada en AWS (MVP en AWS Academy · us-east-1)



Del Entregable 1 al 2: qué se mantuvo, qué cambió y qué falló

Pieza	Entregable 1	Entregable 2	Por qué
Ingesta	API de ingesta en ECS y Kafka 3.6	El productor publica directo en Kafka 4.3.2	La API no tenía un consumidor real (D3). Kafka 3.6 ya no tiene soporte ni imagen oficial
S3 local	MinIO	SeaweedFS 4.52	MinIO dejó de publicar imágenes. Delta pasó la prueba con dos escritores concurrentes (D1)
Log de Delta	S3DynamoDBLogStore	El mismo, con un parche de 5 líneas	Delta 3.3.3 no deja configurar el endpoint de DynamoDB Local
risk_decisions	Escritura con txnAppId dentro del micro-batch	Archivo por micro-batch y commit Delta cada 10 s	Escribir en Delta costaba de 6 a 19 s por micro-batch
Ventanas (RF3)	Dentro del Job 2	Aplicación aparte: job2-ventanas	No quitarle núcleos a la decisión
transaction_uuid	UUID v4	UUID v5 (determinista)	El mismo evento siempre tiene el mismo identificador, también al reprocesar
Alertas a analistas	Correo	Cola SQS sin filtro	No hay servidor de correo en local
Tareas de Airflow	SparkSubmitOperator o DockerOperator	spark-submit local[2] en la imagen de Spark	Sin exponer el socket de Docker; el streaming ocupa los núcleos del worker

Se mantuvo: Kafka, Spark Structured Streaming, MongoDB CP, Delta en tres capas, estrella con SCD 2, cuarentena en Bronze, Trino con catálogo, SNS y SQS, Airflow 3.

Falló: la latencia de la decisión (RNF-02, p99 < 3 s). Se midió de 285 a 53 s. La lámina 11 explica por qué.

Kafka y Spark Structured Streaming, con el modelo en el Job 2

¿Por qué Spark Structured Streaming y no Flink?

- Un solo motor para los 4 jobs, en streaming y en batch, con integración nativa con Delta.
- El equipo ya lo conocía; Flink obligaba a mantener dos motores.
- Costo: latencia = espera del micro-batch + su duración (trigger de 1 s).

¿Por qué Kafka y no Kinesis?

- Spark open source no trae una fuente de Kinesis para Structured Streaming.
- Corre sin internet en Docker; en AWS es MSK, con la misma API.
- Varios consumidores independientes del mismo log (Job 1, Job 2 y sus ventanas), con 7 días de retención.
- Clave source_account: el orden de cada cuenta se conserva en su partición.

Paridad exacta con el notebook (riesgo R8)

58 de 58
variables idénticas al notebook, bit a bit

20.000 de 20.000
decisiones iguales en Kaggle (NumPy 2) y en Docker (NumPy 1.26)

0,26744793
PR-AUC del Job 2 en el test completo, igual al del notebook

Para lograrlo se replicaron las sumas de Kahan de pandas, la aritmética de numexpr (float64 y pow) y el orden por seq dentro del micro-batch. Limitar los offsets por micro-batch rompía la paridad, así que se quitó (D2).

¿Cómo evitamos procesar dos veces un evento?

Kafka entrega al menos una vez. Si Spark falla, repite el micro-batch con los mismos offsets del checkpoint. Por eso **cada efecto es idempotente**.

Componente	Mecanismo	Evidencia
Productor → Kafka	acks=all y enable.idempotence: un reintento no duplica el mensaje	Bronze no repite coordenadas de Kafka
Job 1 → Bronze	Offsets en el checkpoint y sink de Delta: un solo commit por micro-batch	71.910 de 71.910 eventos en Bronze
Job 2 → MongoDB	Cada documento guarda el batch_id que lo escribió y streaming_prev; si el micro-batch se repite, se parte de streaming_prev	Prueba: la escritura falla a la mitad y el reintento deja el mismo estado
Job 2 → risk_decisions	Archivo batch_<id>.parquet con nombre fijo; otra consulta lo agrega a Delta con su propio checkpoint	Una fila por transacción
Job 2 → SNS → entidad	SNS puede repetir; la entidad registra {entidad}:{transaction_uuid} y descarta las repetidas	Probado con moto: la reentrega no se aplica dos veces
Job 3 y Job 4	MERGE por transaction_uuid que solo inserta; Gold lee Silver en una versión fija	Silver sin duplicados; calidad: hechos = Silver

Resultado: **exactly-once en el efecto**, no en el transporte.

Estado en MongoDB (CP) y lakehouse en Delta Lake

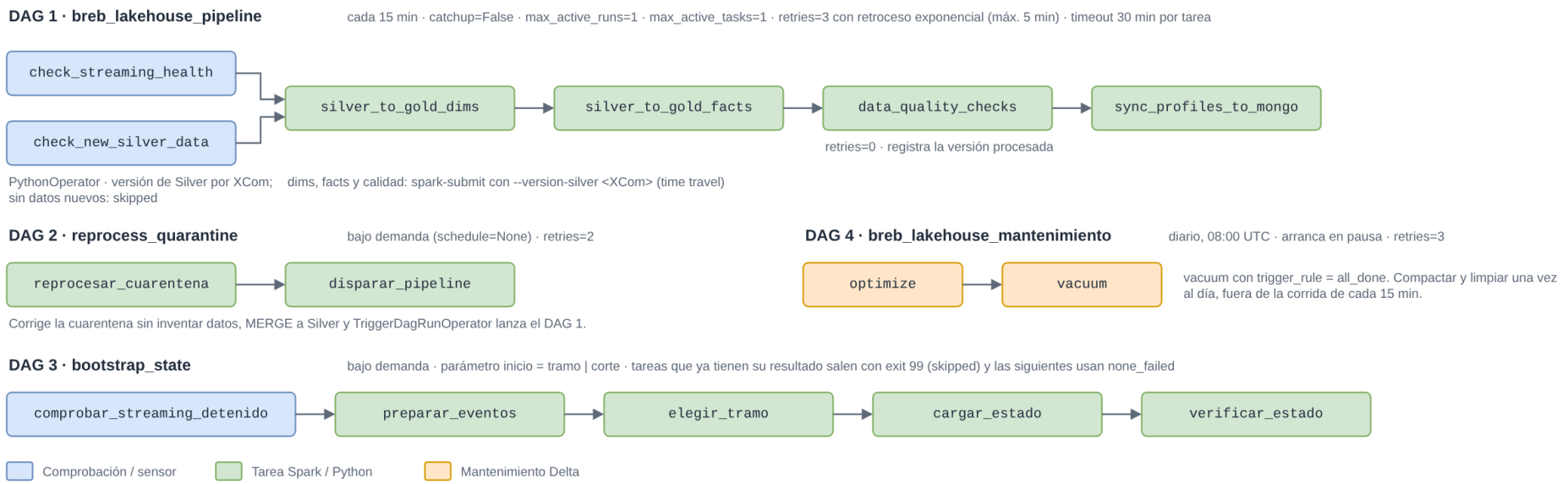
MongoDB 7.0, replica set

- Leer Silver en S3 tarda de 1 a 4 s: no cabe en un micro-batch de 1 s. MongoDB responde por _id, en lote.
- **CP:** write concern y read concern majority. Una escritura confirmada sobrevive a la elección de otro primario.
- **Se sacrifica disponibilidad:** durante la elección (electionTimeoutMillis = 2.000) el Job 2 no escribe y la latencia sube.
- Claves: cuentas._id = cuenta; pares._id = "origen|destino" (dirigido); bancos._id = banco. En producción, sharding por hash de la cuenta.
- Cassandra descartada: sus contadores no son idempotentes ante reintentos.

Delta Lake 3.3 en tres capas

- **Bronze:** append-only, el 100 % de los eventos con sus coordenadas de Kafka; la cuarentena es is_valid = false.
- **Silver:** tipado, en UTC y sin duplicados (MERGE por transaction_uuid).
- **Gold:** estrella con fact_transferencias, dim_cuenta (SCD 2), entidad, canal y tiempo, más risk_decisions y las etiquetas.
- Varios escritores por tabla (Job 3 y la cuarentena en Silver, OPTIMIZE): el log de commits va en DynamoDB.
- Iceberg descartado: más complejo con Structured Streaming en local; Delta trae MERGE, time travel y OPTIMIZE.

Airflow 3 para el batch y Trino con catálogo para consultar



Airflow 3.3.2 (ADR 05)

- Las tres etapas del Job 4 leen Silver en la versión que fija el sensor: un reintento no mezcla datos nuevos.
- La calidad corre antes de escribir en MongoDB. OPTIMIZE y VACUUM van en un DAG diario: eran 8 de los 11 min 39 s.
- Cron descartado: sin dependencias, reintentos ni historial.

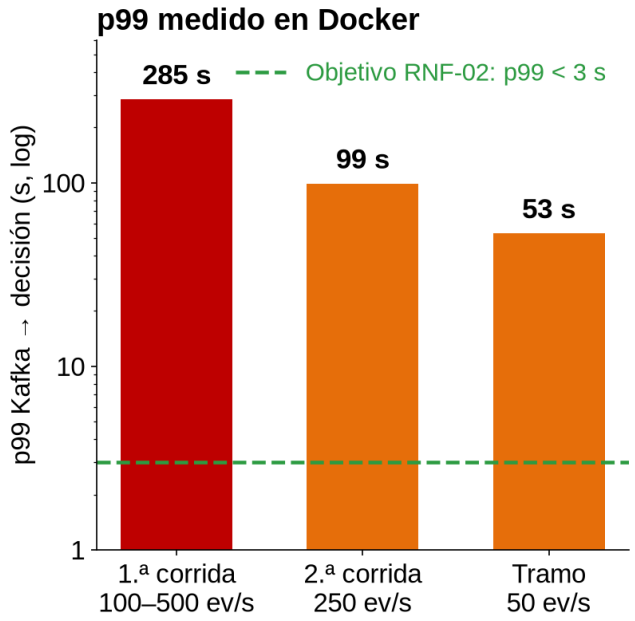
Trino 484 + Hive Metastore (ADR 08)

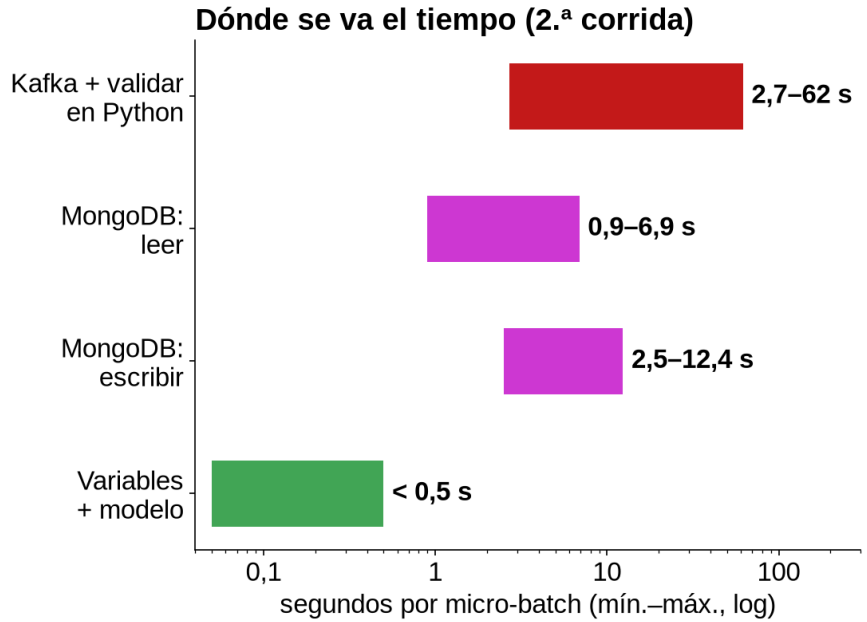
- Una sola capa SQL sobre Bronze, Silver y Gold, que lee el _delta_log (corrección 7).
- Reglas por rol: analista y metabase solo ven Gold; cumplimiento lee las tres capas sin modificar. **10 de 10 casos.**
- Alertas por SNS y una cola SQS por entidad con filtro, en lugar de FastAPI (ADR 09).

Resultados medidos en el equipo de la demo (16 GB, WSL 2)

Requisito	Objetivo	Medido	Estado
RF1 · fuente → Kafka	p99 < 300 ms	p99 de 49 a 235 ms, de 100 a 1.000 ev/s	**Cumple**
RNF-01 · tasa del productor	1.000 ev/s nominales	Los 5 escalones, de 100 a 1.000 ev/s, sin errores	**Cumple**
RNF-02 · Kafka → decisión	p99 < 3 s	285 s → 99 s → 53 s (a 50 ev/s)	**No cumple**
RF5 y RF6 · completitud	100 %, sin duplicados	71.910 de 71.910 en Bronze, Silver y risk_decisions	**Cumple**
R8 · paridad con el notebook	Idéntica	58 de 58 variables; 20.000 de 20.000 decisiones	**Cumple**
RF7 y RF9 · Gold con Airflow	Cada 15 min, orquestado	En Docker: corrida en 3 min 33 s (antes 11 min 39 s); hechos en Gold a los 2 min 21 s. Precisión y recall 0,52	**Cumple**
RNF-06 · linaje	< 5 min	6 s, de Kafka a la acción de la entidad	**Cumple**
RF11 · acceso por rol	Por rol	10 de 10 casos de permisos	**Cumple**
R5 · memoria	Caber en 11,7 GiB	4,0 a 4,6 GiB sin los jobs de streaming; falta medir todo junto	**Parcial**
RNF-03, 04 y 05 · fallas	Bróker, ejecutor, primario	Diseñadas, no medidas	**Pendiente**

Qué falló: la latencia de la decisión, y por qué





- Variables y modelo: **< 0,5 s** por micro-batch.
- El tiempo se va en **leer Kafka y validar el contrato en Python** (2,7 a 62 s) y en **MongoDB** (hasta 12,4 s).
- El equipo está saturado: con los jobs corriendo, el productor vio p99 de 5,5 s hacia Kafka.
- Sacar risk_decisions y las ventanas del camino crítico bajó el p99 de 285 a 99 s.

Otros errores encontrados y su arreglo

Error	Arreglo
Limitar los offsets por micro-batch los repartía entre particiones y cambiaba variables (226 de 9.204 transacciones en la prueba)	Sin tope: el Job 2 ordena cada micro-batch por seq (D2)
SeaweedFS se quedaba sin volúmenes y S3A reintentaba durante minutos	WEED_MASTER_VOLUME_GROWTH_COPY_1=1
docker compose --profile streaming stop apagaba también Kafka, MongoDB y S3	Apagar solo los jobs, por nombre

Qué cambiaríamos con más tiempo, y por qué

#	Cambio	Por qué
1	Validar el contrato con expresiones de Spark (from_json y reglas en SQL) en vez de una pandas UDF con jsonschema	Es el tramo más lento del micro-batch: hasta 62 s
2	Escribir en MongoDB solo los campos que cambian; a escala, estado en el state store de Spark particionado por cuenta	MongoDB suma hasta 12,4 s por micro-batch; acercar el estado al cómputo quita viajes de red
3	Medir en Linux nativo o en una nube con más núcleos y declarar la tasa con 30 % de margen	El portátil está saturado (el productor ve 5,5 s hacia Kafka) y AWS Academy da 2 vCPU por instancia
4	Correr las pruebas de falla: bróker, ejecutor de Spark y elección del primario de MongoDB	RNF-03, RNF-04 y RNF-05 están diseñados, pero no medidos
5	Gold incremental con el Change Data Feed de Silver	Hoy el Job 4 compara contra todos los hechos, y eso crece con el histórico
6	Secrets Manager, KMS con llave propia, autenticación en Trino y CI con las pruebas de paridad	En AWS ya hay rol de IAM, HTTPS y cifrado por defecto; faltan los secretos y la identidad en Trino

Demo en vivo: qué vamos a mostrar (8 minutos)

0:00

Estado de partida

Estado inicial en MongoDB (bootstrap_state) y la primera parte del tramo ya en Gold

0:40

Productor en vivo

El resto del tramo, a 50 ev/s y en orden de seq

1:00

Streaming

Spark UI y log del Job 2 por micro-batch; alertas a la entidad y a los analistas

3:45

Airflow

El DAG principal: sensores, dimensiones, hechos, calidad y perfiles

6:15

Gold

Consulta en Trino o Metabase: alertas por canal y desempeño contra las etiquetas

7:15

Linaje

Una alerta de esta corrida, de Kafka a la acción de la entidad

En pantalla

- Navegador: Spark (localhost:8080), Airflow (localhost:8085), Metabase (localhost:3000) y el repositorio en GitHub.
- Terminales: productor, log del Job 2, entidad, analistas y Trino.
- Si algo falla, lo explicamos con la evidencia en pantalla: qué falló, por qué y cómo se arregla.

Respaldo: si el volumen se multiplicara por 100

Referencia: el promedio del día récord son 120 ev/s; por 100 son **12.000 ev/s**, unos 12 MB/s con eventos de 1 KB. Kafka no es el cuello; el procesamiento con estado sí.

Pieza	Hoy	Qué cambiaríamos (en orden)
1 · Job 2	Validación en Python y estado en MongoDB en cada micro-batch: el cuello medido	Validación con expresiones de Spark; estado en el state store (RocksDB) particionado por cuenta; MongoDB solo para perfiles y consultas
2 · Kafka	6 particiones, 1 o 3 brókers	48 a 96 particiones, definidas antes de producción (cambiarlas reasigna las claves); MSK en 3 zonas
3 · MongoDB	Replica set sin sharding	Sharding por hash de la cuenta; escribir solo los campos que cambian
4 · Spark	1 worker de 5 núcleos para los 4 jobs	Un clúster por job con autoescalado (EMR o Kubernetes); shuffle.partitions queda fijo en el checkpoint
5 · Lakehouse	risk_decisions y Gold sin particionar	Particionar por fecha o usar liquid clustering; OPTIMIZE por partición
6 · Job 4	Anti-join de Silver contra todos los hechos	Change Data Feed de Silver: procesar solo lo nuevo
7 · Consumo	Trino de un nodo	Athena o un clúster de Trino

Respaldo: qué pasa si algo falla

Falla	Qué pasa	En el código
Cae un bróker de Kafka	Perfil completo (RF=3, min.insync.replicas=2): acks=all sigue escribiendo. Con un bróker (dev): el productor reintenta hasta 5 min, los jobs fallan, Docker los reinicia y siguen desde el checkpoint	publicador.py:143 · crear_topicos.sh:10-14 · docker-compose.yml:79
Spark falla a mitad de un micro-batch	Repite el micro-batch con los mismos offsets; MongoDB parte de streaming_prev; Delta y MERGE no duplican	comun.py:74-81 · almacen.py:34-80 · test_job2_microbatch.py:96
Cae el primario de MongoDB	Se elige otro primario (timeout de 2 s); el Job 2 espera y la latencia sube; no se pierde ninguna escritura confirmada	iniciar_replica.js:34 · almacen.py:30
SNS no responde	El micro-batch falla y se repite; puede haber alertas duplicadas, que la entidad descarta	job2_riesgo.py:112-128 · servicio.py:78-101
Falla una tarea de Airflow	Hasta 3 reintentos con retroceso exponencial, sobre la misma versión de Silver	dags/breb_lakehouse_pipeline.py:43
Llegan eventos inválidos	Quedan en Bronze con is_valid = false; reprocess_quarantine corrige lo que puede y hace MERGE a Silver	job1_bronze.py · batch/cuarentena.py:49
Se acaba la memoria	Docker reinicia el contenedor (restart: unless-stopped) y el job sigue desde el checkpoint	topes por servicio en infra/docker-compose.yml